Object-Oriented Programming and Data Structures

COMP2012: Namespace

Cecia Chan
Brian Mak
Pedro Sander

Department of Computer Science & Engineering
The Hong Kong University of Science and Technology
Hong Kong SAR, China

Motivation

Suppose that you want to use two libraries, each consisting of a bunch of useful classes and functions, but some of them have the same name.

```
/* File: apple-utils.h */
class Stack      { /* incomplete */ };
class Some_Class { /* incomplete */ };
void safari()    { cout << "Apple's browser" << endl; };
void app(int x)  { cout << "Apple's app: " << x << endl; };
```

```
/* File: ms-utils.h */
class Stack       { /* incomplete */ };
class Other_Class { /* incomplete */ };
void edge()       { cout << "Microsoft's browser" << endl; };
void app(int x)   { cout << "Microsoft's app: " << x << endl; };
```

Motivation ..

Even if you don't use Stack and app, you run into troubles:

- compiler complains about multiple definitions of Stack;
- compiler/linker complains about multiple definitions of app.

```
1  #include <iostream>     /* File: use-utils.cpp */
2  using namespace std;
3  #include "apple-utils.h"
4  #include "ms-utils.h"
5  enum class OS { MSWindows, MacOS } choice;
6
7  int main()
8  {
9      Some_Class sc;
10     Other_Class oc;
11
12     if (choice == OS::MacOS)
13         safari();
14     else if (choice == OS::MSWindows)
15         edge();
16     return 0;
17 }
```

Solution: namespace

Solution: namespace ..

If the library writers would have used namespaces, multiple names wouldn't be a problem.

```
/* File: apple-utils-namespace.h */
namespace apple
{
    class Stack      { /* incomplete */ };
    class Some_Class { /* incomplete */ };
    void safari()    { cout << "Apple's browser" << endl; };
    void app(int x)  { cout << "Apple's app: " << x << endl; };
}
```

```
/* File: ms-utils-namespace.h */
namespace microsoft
{
    using namespace std;
    class Stack       { /* incomplete */ };
    class Other_Class { /* incomplete */ };
    void edge()       { cout << "Microsoft's browser" << endl; };
    void app(int x)   { cout << "Microsoft's app: " << x << endl; };
}
```

Namespace Alias & Scope Operator ::

Refer names in a namespace with the scope resolution operator.

```
1  #include <iostream>                    /* File: utils-namespace.cpp */
2  using namespace std;
3  #include "ms-utils-namespace.h"
4  #include "apple-utils-namespace.h"
5  namespace ms = microsoft;              // Namespace alias
6  enum class OS { MSWindows, MacOS } choice;
7
8  int main()
9  {
10     apple::Some_Class sc; apple::Stack apple_stack;
11     ms::Other_Class oc; ms::Stack ms_stack;
12     ms::app(42);
13
14     cout << "Input your OS choice: ";
15     int int_choice; cin >> int_choice; // Can't cin to choice. Why?
16     switch (choice = static_cast<OS>(int_choice))
17     {
18         case OS::MSWindows: ms::edge(); break;
19         case OS::MacOS: apple::safari(); break;
20         default: cerr << "Unsupported OS" << endl;
21     }
22     return 0;
23 }
```

using Declaration

If you get tired of specifying the namespace every time you use a name, you can use a using declaration.

```
1  #include <iostream>        /* File: utils-using.cpp */
2  using namespace std;
3  #include "ms-utils-namespace.h"
4  #include "apple-utils-namespace.h"
5  namespace ms = microsoft; // Namespace alias
6  using apple::Some_Class;
7  using ms::Other_Class;
8  using apple::Stack;
9  using ms::app;
10
11 int main()
12 {
13     Some_Class sc;         // Refer to apple::Some_Class
14     Other_Class oc;        // Refer to ms::Other_Class
15     Stack apple_stack;     // Refer to apple::Stack
16     ms::Stack ms_stack;
17     app(2); return 0;      // Refer to ms::app
18 }
```

Ambiguity With using Declarations

You can also bring all the names of a namespace into your program at once, but make sure it won't cause any ambiguities.

```
1  #include <iostream>                /* File: utils-using-err.cpp */
2  using namespace std;
3  #include "ms-utils-namespace.h"
4  #include "apple-utils-namespace.h"
5
6  namespace ms = microsoft;          // Namespace alias
7  using namespace apple;
8  using namespace ms;
9
10 int main()
11 {
12     Some_Class sc;                 // Refer to apple::Some_Class
13     Other_Class oc;                // Refer to ms::Other_Class
14     Stack S;                       // Error: ambiguous;
15     ms::Stack ms_stack;            // OK
16     apple::Stack apple_stack;      // OK
17     return 0;
18 }
```

Namespace std

```
1   #include <iostream>     /* File: std-using.cpp */
2   using namespace std;
3
4   int main()
5   {
6       string s;
7       cin >> s;
8       cout << s << endl;
9
10      s += " is good!";
11      cout << s << endl;
12
13      return 0;
14  }
```

How Should We Declare Namespaces?

- Functions and classes of the standard library (`string`, `cout`, `isalpha()`, ...) and the STL (`vector`, `list`, `foreach`, `swap`, ...) are all defined in namespace std.
- Here, we bring all the names that are declared in the three header files into the global namespace.
- Although the previous program works, it is considered bad practice to declare the namespace std globally.
- It is better to introduce only the names you really need, or to qualify the names whenever you use them.
- Although this takes more typing effort, it is also immediately clear which functions and classes are from the standard (template) library, and which are your own.
- A combination of using declarations and explicit scope resolution is also possible; this is mostly a matter of taste.

Explicit Use of using Declaration Per Object/Function

```
1   #include <iostream>     /* File: std-per-obj-using.cpp */
2   using std::string;
3   using std::cin;
4   using std::cout;
5   using std::endl;
6
7
8   int main()
9   {
10      string s;
11      cin >> s;
12      cout << s << endl;
13
14      s += " is good!";
15      cout << s << endl;
16
17      return 0;
18  }
```

Explicit Use of namespace Per Instance of Object/Function

```
1   #include <iostream>     /* File: std-per-instance-using.cpp */
2   using namespace std;
3
4   int main()
5   {
6       std::string s;
7       std::cin >> s;
8       std::cout << s << std::endl;
9
10      s += " is good!";
11      std::cout << s << std::endl;
12
13      return 0;
14  }
```

Namespace Is Expansible

- Namespaces can be defined in steps and nested.

```
1  #include <iostream>     /* File: misc-namespace.cpp */
2
3  namespace hkust
4  {
5      namespace cse { int rank() { return 1; } } // Nested namespace
6      void good() { std::cout << "Good!" << std::endl; }
7  }
8
9  namespace hkust   // Extend the namespace
10 {
11     void school() { std::cout << "School!" << std::endl; }
12 }
13
14 int main()
15 {
16     std::cout << "CSE's rank: " << hkust::cse::rank() << std::endl;
17     hkust::good();
18     hkust::school(); return 0;
19 }
```